

DESARROLLO WEB HIGH-END

Web 100/100: Animaciones Canvas & SVGator

Crea experiencias web fascinantes con animaciones vectoriales SVGator, canvas de alto rendimiento, diseño cyberpunk neón y optimización SEO técnica.

[Nivel: Todos los niveles]

[Duración: 10hs]

[8 Módulos]

[Tech: SVGator, HTML5 Canvas, CSS3]

#> Módulo 1: UX/UI Cyberpunk Neón

El diseño 'Cyberpunk Neón' se basa en fondos muy oscuros (casi negros) contrastados con colores hiper-saturados (magenta, cyan, verde lima).

El Secreto del Brillo Neón

Para crear un texto o caja que parezca realmente iluminada, no basta con un solo 'box-shadow'. Debemos apilar múltiples sombras de CSS para emular cómo la luz se difumina en el aire.

```
.neon-button {
  background: transparent;
  color: #ff0099;
  border: 2px solid #ff0099;
  /* Apilamos 4 sombras: contorno duro, brillo medio, brillo extendido y resplandor lejano */
  box-shadow:
    0 0 5px #ff0099,
    0 0 15px #ff0099,
    0 0 30px #ff0099,
    inset 0 0 10px #ff0099;
}

.neon-text {
  text-shadow: 0 0 10px #00ffff, 0 0 20px #00ffff;
}
```

Al usar tipografías monospace (como Roboto Mono o Fira Code), completas la ilusión de una terminal futurista.

#> Módulo 2: Maestría en CSS3 Keyframes

Antes de saltar a Canvas o librerías pesadas, domina el CSS. La regla de oro del rendimiento web es: anima SOLAMENTE propiedades que usen aceleración por hardware (GPU).

El Peligro del Reflow

Si animas propiedades como ``width``, ``height``, ``top`` o ``margin``, obligas al navegador a recalcular el diseño de TODA la página 60 veces por segundo. Esto destruye la batería de los celulares.

```
/* MAL: Usa CPU y causa tirones (Lag) */
.caja { animation: mover-mal 2s infinite; }
@keyframes mover-mal {
  to { left: 200px; }
}

/* EXCELENTE: Usa GPU y corre a 60fps constantes */
.caja { animation: mover-bien 2s infinite; }
@keyframes mover-bien {
  to { transform: translateX(200px); }
}
```

Siempre usa ``transform`` (para mover, escalar o rotar) y ``opacity`` (para desvanecer).

#> Módulo 3: Introducción a SVGator

SVG (Scalable Vector Graphics) es código matemático que dibuja formas. Nunca pierde calidad al hacer zoom. SVGator es una herramienta visual (como After Effects) que genera código SVG animado sin necesidad de programarlo a mano.

El Flujo de Trabajo en SVGator

1. Dibuja tus assets en Illustrator/Figma (ej. un cohete).
2. Importa el archivo .svg a SVGator.
3. Usa la línea de tiempo (Timeline) para crear keyframes de opacidad, rotación y trazado (Path Animation).
4. Exporta tu proyecto. SVGator inyectará el CSS o JavaScript directamente dentro del archivo SVG resultante.

#> Módulo 4: Integración Web de SVGator

Una vez exportado, no uses un tag `<img>` tradicional si quieres interactividad.

Inyectando SVG y Controlándolo con JS

Si exportas en modo JavaScript, SVGator incluye una pequeña API interna.

```
<!-- Pega el código SVG directamente en tu HTML (Inline SVG) -->
<svg id="mi-animacion" viewBox="0 0 800 600">
  <!-- Paths... -->
</svg>

<script>
  // Cuando el usuario haga hover en el contenedor, reproducimos la animación
  const svgElement = document.getElementById('mi-animacion');
  svgElement.addEventListener('mouseenter', () => {
    // SVGator expone métodos si se exportó correctamente
    svgElement.svgatorPlayer.play();
  });
</script>
```

Esto permite crear íconos que reaccionan de manera ultra-suave a las interacciones del usuario, pesando apenas unos pocos kilobytes.

#> Módulo 5: El Poder de HTML5 Canvas

El elemento `<canvas>` proporciona una superficie en la que puedes dibujar gráficos mediante JavaScript. A diferencia del SVG (que guarda un objeto en memoria por cada línea), Canvas dibuja 'píxeles muertos', lo que permite renderizar decenas de miles de objetos simultáneamente sin crashear el navegador.

Tu Primer Contexto 2D

```
<canvas id="lienzo" width="800" height="600"></canvas>

<script>
  const canvas = document.getElementById('lienzo');
  const ctx = canvas.getContext('2d');

  // Dibujar un rectángulo color Neón
  ctx.fillStyle = '#ff0099';
  ctx.shadowBlur = 20;
  ctx.shadowColor = '#ff0099';
  ctx.fillRect(100, 100, 200, 100);
</script>
```

#> Módulo 6: Alto Rendimiento (requestAnimationFrame)

Para crear animaciones en Canvas (ej. nieve cayendo o partículas de código Matrix), necesitas borrar y volver a dibujar el lienzo 60 veces por segundo.

El Bucle Infinito Perfecto

Jamás uses `setInterval(draw, 16)`. No está sincronizado con el monitor del usuario y causará fragmentación de imagen (tearing).

```
let x = 0;

def update() {
  // 1. Borrar pantalla
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  // 2. Dibujar frame actual
  ctx.fillStyle = '#00ff41';
  ctx.fillText("010110", x, 50);

  // 3. Actualizar lógica
  x += 2;

  // 4. Pedirle al navegador que llame a update() justo antes de dibujar la pantalla
  requestAnimationFrame(update);
}

// Iniciar bucle
update();
```

`requestAnimationFrame` se pausa automáticamente cuando el usuario cambia de pestaña, ahorrando un 100% de batería y CPU.

#> Módulo 7: Partículas Interactivas

Las webs premium 'sienten' al usuario. Haremos que cientos de partículas en Canvas reaccionen al mouse.

Física de Partículas Básica

```
const mouse = { x: null, y: null };
window.addEventListener('mousemove', (event) => {
  mouse.x = event.x;
  mouse.y = event.y;
});

class Particula {
  constructor(x, y) { this.x = x; this.y = y; this.size = 3; }

  update() {
    // Calcular distancia entre el mouse y la partícula
    let dx = mouse.x - this.x;
    let dy = mouse.y - this.y;
    let distancia = Math.sqrt(dx * dx + dy * dy);

    if (distancia < 50) {
      // Si el mouse está cerca, repeler la partícula
      this.x -= dx / 10;
      this.y -= dy / 10;
    }
    this.draw();
  }

  draw() { /* ctx.arc(...) */ }
}
```

Llenarás un Array con miles de estas partículas y ejecutarás `update()` en tu bucle `requestAnimationFrame`. El resultado es magia pura.

#> Módulo 8: SEO Técnico y PageSpeed 100/100

Google castiga severamente las páginas lentas. ¿Cómo logramos PageSpeed 100/100 con tantas animaciones?

Lazy Loading Estratégico

1. Carga diferida de scripts: Si tu script de Canvas está al final de la página (fuera del 'Above the Fold'), no lo cargues al inicio. Usa el atributo `defer` o usa `IntersectionObserver` para iniciarlo solo cuando el usuario haga scroll hasta él.
2. SVG en línea: Los SVGs insertados directamente en el HTML no requieren peticiones HTTP adicionales, lo que elimina latencia de red.
3. Fuentes Modernas: Usa la propiedad CSS `font-display: swap;` para asegurar que el texto sea visible inmediatamente mientras la fuente Neón futurista se descarga de fondo.

¡Has completado el curso! Ahora tienes la capacidad de crear experiencias web que no solo deslumbran visualmente, sino que pasan cualquier auditoría de rendimiento estricta.

CREATIVE FRONTEND DEVELOPER

Certificación otorgada a:

[CREATIVE DEVELOPER]

Por haber dominado el arte de la web interactiva, combinando físicas de partículas en Canvas, animaciones vectoriales SVG y estéticas UI/UX avanzadas manteniendo un PageSpeed Score de 100/100.

TIMESTAMP: 2026-08-12 // CREATIVE_DIRECTOR_SIGNATURE